Python for Artificial Intelligence

The ML Workflow & Linear Regression

Week 4, Day 2

Today's Project: A Bykea/Careem Fare Estimator

- This week you build one running project: a ride-hailing app's brain, piece by piece
- Today: predict a ride's fare from its distance, the time of day, and the area, using real ML, not a fixed price list
- Hour 1: supervised learning, regression, and the full ML workflow
- Hour 2-3: Linear Regression, how it is fit, how to read it, and how to judge how good it actually is

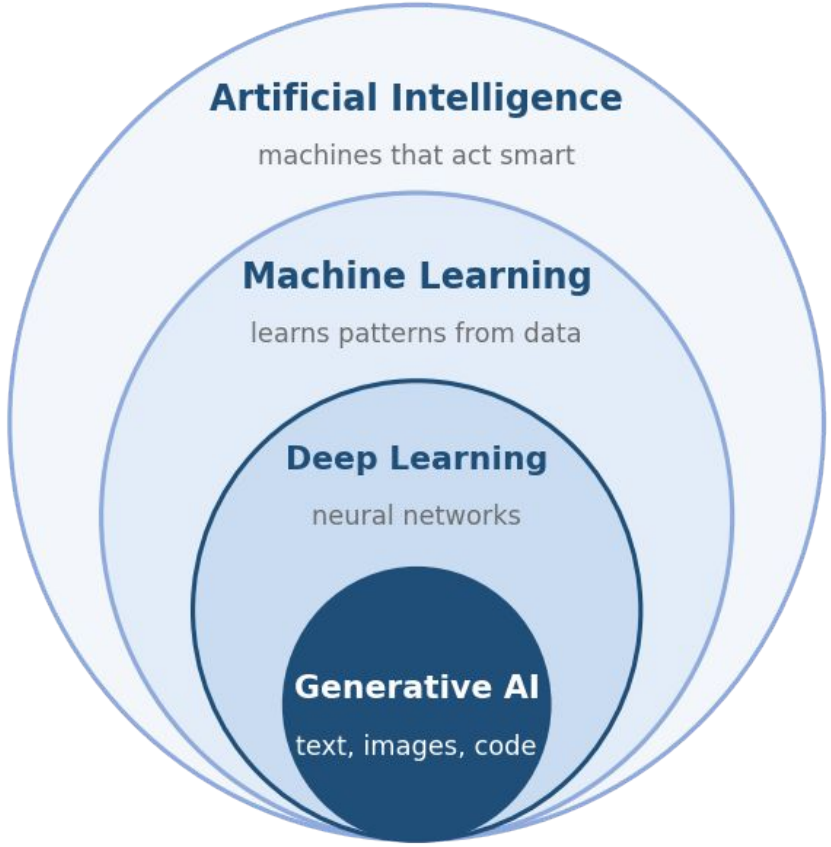
Hour 1

Supervised Learning & the ML Workflow

CONCEPT

The Domains of AI

- Artificial Intelligence (AI) is the big umbrella: any machine doing something that normally needs human intelligence
- Machine Learning (ML) is the part of AI that learns from data instead of following hand-written rules
- Deep Learning (DL) is the part of ML that uses many-layered neural networks
- Generative AI is deep learning that creates new text, images and code. Chatbots live here
- Each ring sits inside the one before it: all deep learning is ML, and all ML is AI, but not the other way round



REFERENCE

AI vs ML vs Deep Learning: Key Differences

Same family, different levels of automation and data hunger.

	AI	Machine Learning	Deep Learning
Idea	Machines that act intelligently	Machines that learn from data	Learning with many-layered neural networks
Rules come from	Humans, or learned	Learned from examples	Learned automatically, layer by layer
Data needed	Varies	Hundreds to thousands of rows	Often millions of examples
Typical input	Anything	Tables of features	Images, audio, text
Hardware	Ordinary computer	Ordinary computer	Often needs a GPU
Ride example	A full ride-hailing app	Predict a ride's fare (this week)	Read a pickup photo, or Day 19-20 neural net

CONCEPT

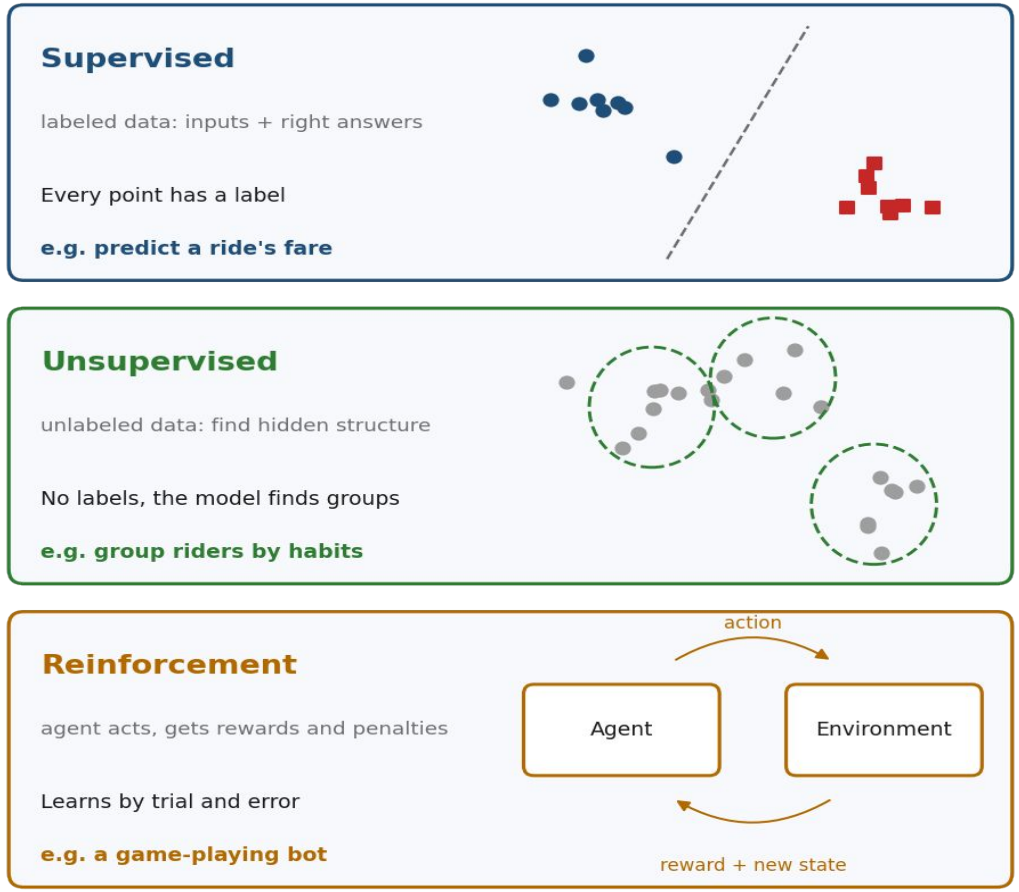
What Is Machine Learning?

- Traditional programming: a human writes explicit rules, the computer follows them exactly
- Machine learning: the computer learns the rules itself, by finding patterns in examples (data)
- A fixed rule like "fare = 60 + 20 per km" is traditional programming. Learning that formula from thousands of real past rides is ML
- Every model you build this month, from Linear Regression to neural networks, follows this same idea
- Boil it down and most ML does one important job: prediction. It studies past examples, finds the pattern, and makes a good guess about something new

CONCEPT

Three Types of Machine Learning

- Supervised > prediction: learn from examples that come with the right answer. Predict a fare, flag spam
- Unsupervised > patterns: no answers given, the model finds structure on its own. Group similar riders, spot odd rides
- Reinforcement > decisions: an agent learns what to do next by trial and error, collecting rewards and penalties. Game bots, robots, self-driving
- Quick memory line: supervised predicts, unsupervised finds patterns, reinforcement learning makes decisions
- This course spends most of its time on supervised learning, because that is what most real business ML is



REFERENCE

Supervised vs Unsupervised vs Reinforcement

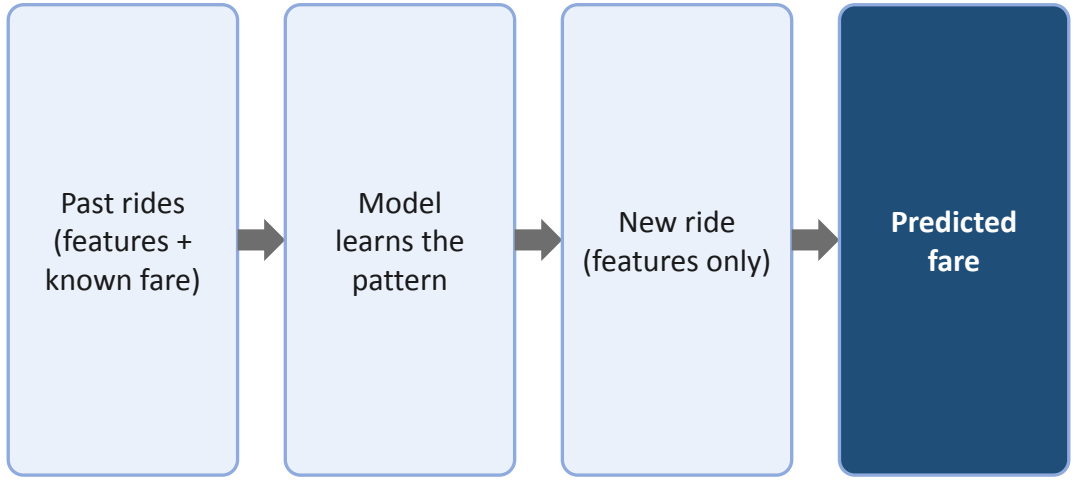
The question to ask: what feedback does the model get?

	Supervised	Unsupervised	Reinforcement
Core job	Prediction	Finding patterns	Making decisions
Data	Labeled (inputs + answers)	Unlabeled (inputs only)	No dataset: an environment
Feedback	Correct answer for every example	None	Reward or penalty after actions
Goal	Predict a number or category	Find groups and patterns	Learn the best actions over time
Examples	Fare prediction, spam filter, loan approval	Customer segments, anomaly detection	Game bots, robot control, route optimization
Ride app use	Predict fare, predict quick pickup	Group riders by habits	Dynamically decide driver incentives

CONCEPT

Supervised Learning: Learning From Labeled Examples

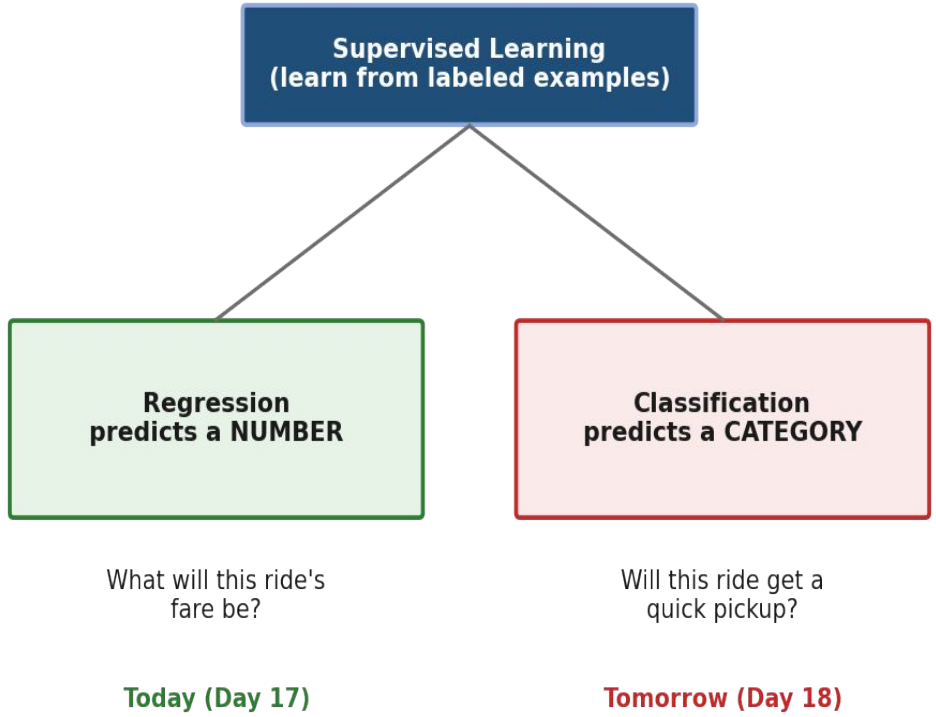
- Supervised learning trains a model on data where the right answer is already known, then predicts it for new, unseen cases
- Input (features): what we know about a ride before it starts, like distance and time of day
- Output (target or label): what we want to predict, like the fare
- The model learns the link between the two from past rides, then applies it to a brand-new request



CONCEPT

Two Types: Regression and Classification

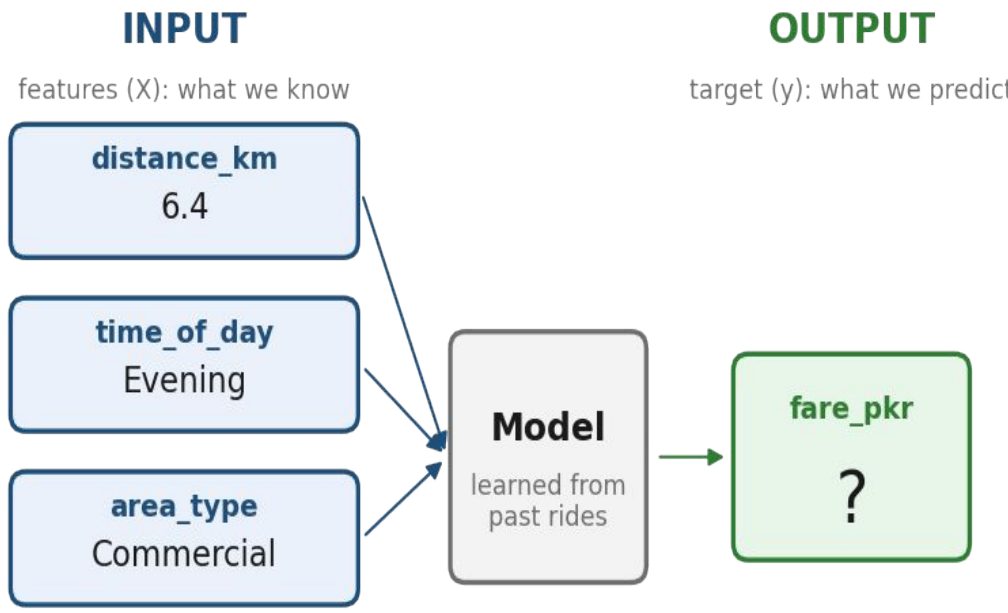
- Regression predicts a continuous number: a fare, a price, a temperature, a score
- Classification predicts a category: quick pickup or long wait, spam or not spam
- The first question for any ML problem: is the target a number or a category?
- Today is regression. Tomorrow is classification, on the very same ride data



CONCEPT

Input vs Output: Features and Target

- Features are the INPUT: everything we know about a ride before it starts (also called X)
- The target is the OUTPUT: the one thing we want the model to predict (also called y)
- Features go in, the target comes out. The model is the link between them
- Same data, different target: predict fare_pkr and it is regression, predict quick_pickup and it is classification
- A feature must be known at prediction time. Using the fare to predict pickup would be cheating (data leakage)



Features go in. The target comes out.

REFERENCE

Regression Problems: Features and Target

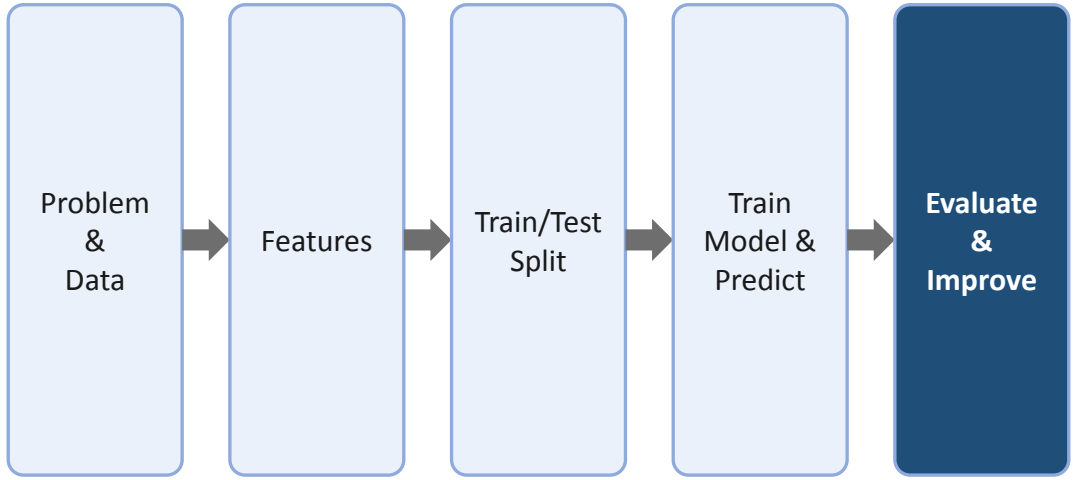
Every regression problem has the same shape: some measurable features, and a number to predict.

Problem	Features (Input, X)	Target (Output, y)
Predict a ride's fare	Distance, time of day, area	Fare (PKR)
Predict monthly house rent	Size, bedrooms, locality	Rent (PKR)
Predict a used phone's price	Age, storage, condition	Resale price (PKR)
Predict a student's exam score	Hours studied	Exam score

CONCEPT

The Complete ML Workflow

- Every ML project, no matter how advanced, follows the same sequence of stages
- Problem: predict a ride's fare, before the ride starts, from details we already know
- Data & Features: collect past rides, and decide which columns actually help predict fare
- Model, Evaluate, Improve: train a model, measure how well it works, then refine it



PROJECT

Meet the Dataset: ride_requests.csv

A log of past ride requests, the raw material for this week's entire project.

Column	Meaning	Example
distance_km	Distance from pickup to drop-off	6.4
time_of_day	Morning, Afternoon, Evening, or Night	Evening
area_type	Residential, Commercial, or Airport	Commercial
fare_pkr	What the rider was actually charged	245
quick_pickup	1 if a driver accepted within 3 minutes	1

CONCEPT

Why Split Data Into Train and Test Sets?

- A model is trained on one portion of past rides (the training set) and judged on rides it has never seen (the test set)
- If you evaluate a model on the same rides it trained on, you're testing its memory, not its ability to predict a brand-new ride
- A typical split is 80% training, 20% testing, chosen randomly so both sets look like the full dataset
- Good fare predictions on the test set are the real evidence the model learned a genuine pattern, not a coincidence

EXAMPLE

Splitting ride_requests.csv

scikit-learn's train_test_split does this in one line.

```
import pandas as pd
from sklearn.model_selection import train_test_split

rides = pd.read_csv('ride_requests.csv')
X = rides[['distance_km']]
y = rides['fare_pkr']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

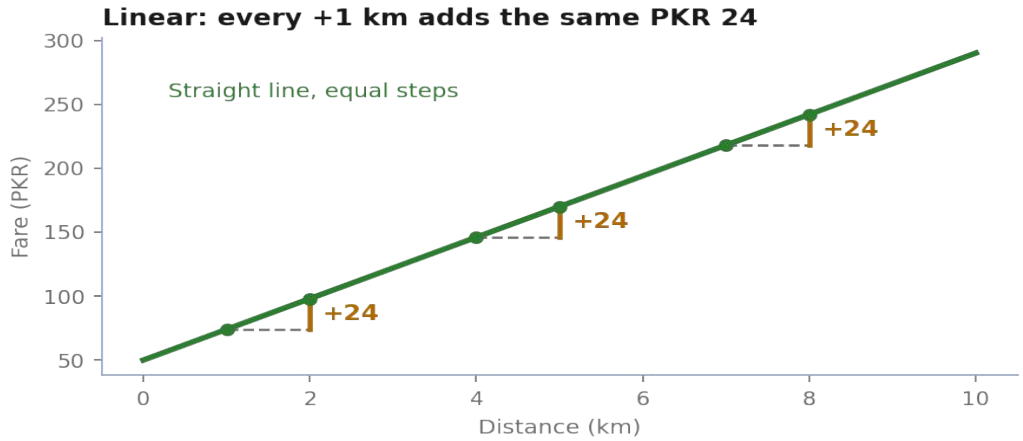
Hour 2

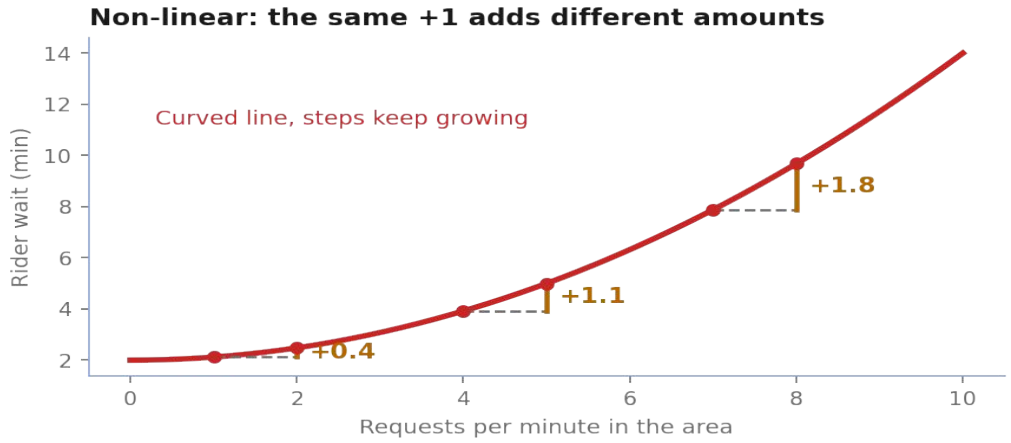
Linear Regression: Fitting a Fare Estimator

CONCEPT

Linear vs Non-Linear Relationships

- A linear relationship means: when X goes up by 1 unit, Y changes by the same fixed amount, wherever you start
- Fare example: each extra kilometer adds about PKR 24, whether the ride grows from 2 to 3 km or from 20 to 21 km
- On a graph, that gives a straight line: equal steps, equal rises
- Non-linear means the change depends on where you are. Rider wait time grows slowly at first, then faster as the area gets busier
- The slope (coefficient) is that fixed amount, so one number describes the whole linear relationship
- Real data is never perfect: "roughly linear" means the points scatter around a straight line

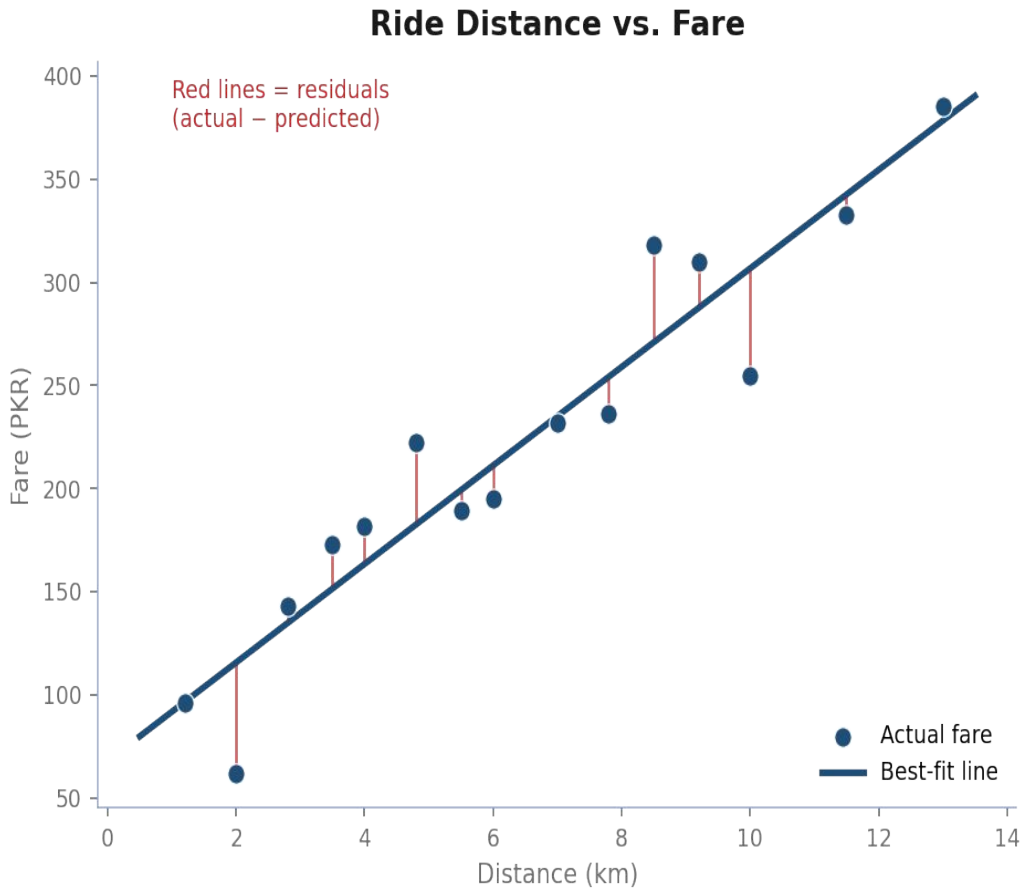




CONCEPT

Linear Regression, Intuitively

- Linear Regression draws the single straight line that best fits a scatter of points, here: distance vs. fare
- "Best" means the line minimizes the total error between its predictions and the actual fares, the cost function
- The gap between an actual fare and the line's predicted fare, for one ride, is called a residual
- Once fit, the line itself is the model: plug in a distance, read off a predicted fare



CONCEPT

The Equation, Annotated

- Predicted fare = intercept + (coefficient x distance) + error
- The intercept is the baseline fare. Each coefficient says how much fare changes per unit of its feature
- The error term is whatever the straight line can't explain, the residual
- With several features, the equation just adds one more coefficient-times-feature term for each

One feature (distance)

$$\hat{y} = b_0 + b_1 \times x + \varepsilon$$

predicted fare | intercept (base fare) | coefficient (PKR per km) | input (distance) | error (what's left)

Several features (distance, time, area)

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \ldots + \varepsilon$$

distance x its coefficient | time of day x its coefficient

Each feature gets its own coefficient: its own readable effect on fare

CHECKLIST

Is It a Regression Problem? Questions 1 and 2

- Question 1: Is the target a number? Fare, price, temperature, score. If it is a category, use classification instead
- Question 2: Are the inputs measurable? Numbers like distance work directly
- Categorical inputs like time_of_day must be converted to numbers first, using one-hot encoding
- Never label-encode a category as 1, 2, 3 for regression: it falsely implies Night is three times Morning

Label encoding (misleading)

time_of_day	code
Morning	1
Evening	2
Night	3

Implies Night = 3x Morning. A regression would treat it as a real quantity.

One-hot encoding (correct)

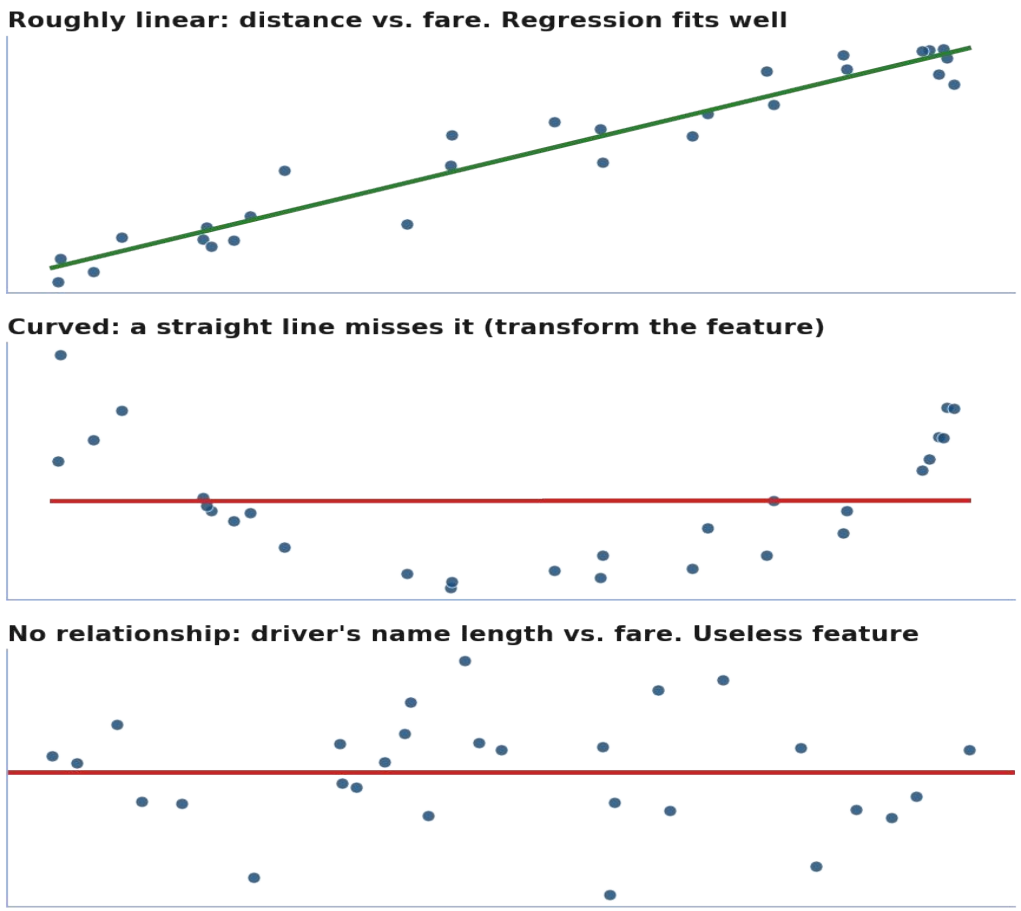
time_of_day	Evening	Night
Morning	0	0
Evening	1	0
Night	0	1

Each category gets its own 0/1 column. Morning is the baseline (both columns 0).

CHECKLIST

Is It a Regression Problem? Question 3

- Question 3: do you expect a roughly linear relationship? If distance goes up, does fare go up proportionally?
- Hint: plot the target against each feature. If it looks roughly like a line, regression will work
- A curved relationship can still work if you transform the feature first (square it, take a log)
- A feature with no pattern at all adds nothing, and can even hurt the model



CHECKLIST

Is It a Regression Problem? Question 4

- Question 4: do you want to understand what drives the number, not just predict it?
- Linear Regression gives one coefficient per feature, a readable statement of that feature's effect
- "Each extra kilometer adds about PKR 24, holding everything else constant" is a sentence a manager can act on
- That readability is the main reason to choose Linear Regression over a more complex, opaque model

EXAMPLE

See the Line First: seaborn regplot

regplot fits a straight line (the same least-squares method) and draws it over the scatter. Great for looking, but it gives you no model to predict with.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.regplot(x='distance_km', y='fare_pkr', data=rides)
plt.show()

# The shaded band is a confidence interval. Hide it with ci=None
# Next: LinearRegression gives the same line AS A MODEL we can use
```

EXAMPLE

Fitting the First Fare Estimator

One feature (distance), one line. Four lines of code is the entire model.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print(model.intercept_)   # base fare, about PKR 50
print(model.coef_)        # about PKR 23.5 per km
```

EXAMPLE

Plotting the Fitted Line on Our Data

Scatter the real test rides, then draw the model's predictions as a line over them.

```
import matplotlib.pyplot as plt
import numpy as np

# 1. The actual rides
plt.scatter(X_test['distance_km'], y_test, label='Actual rides')

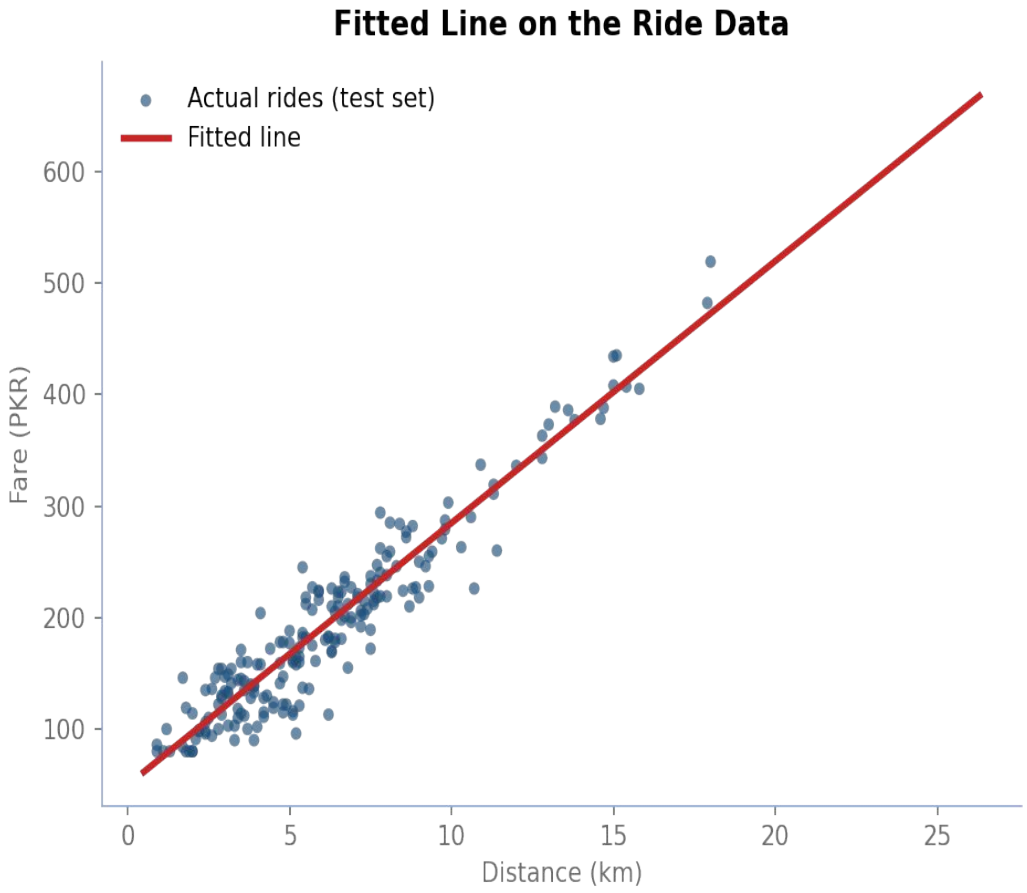
# 2. Predictions for evenly spaced distances give the fitted line
line_x = pd.DataFrame({'distance_km': np.linspace(X['distance_km'].min(),
                                                  X['distance_km'].max(), 100)})
plt.plot(line_x, model.predict(line_x), color='red', label='Fitted line')

plt.xlabel('Distance (km)')
plt.ylabel('Fare (PKR)')
plt.legend()
plt.show()
```

RESULT

The Fitted Line on the Ride Data

- Each blue dot is a real test ride. The red line is the model's predicted fare for every distance
- The line climbs about PKR 24 for each extra kilometer, the coefficient we just read
- Dots above the line are rides that cost more than predicted, dots below cost less
- The vertical gap between a dot and the line is that ride's error, the very thing MAE and MSE measure
- Compare it to seaborn's regplot: the same line, but now it comes from a model we can predict with



CONCEPT

Reading the Model's Coefficients

- The intercept is the predicted fare at distance = 0: a rough "base fare" the model has learned on its own
- The coefficient is the predicted PKR added per extra kilometer: the model's learned "per-km rate"
- Neither number was programmed in, both were estimated purely from past ride data
- A model you can read like this, in plain language, is one of Linear Regression's biggest advantages over more complex models

EXAMPLE

Making Predictions With model.predict()

Pass new rides in the same shape as the training features: a DataFrame with a distance_km column.

```
import pandas as pd

new_rides = pd.DataFrame({'distance_km': [3, 6, 12]})
print(model.predict(new_rides))

# [120.4  190.8  331.7]
# 3 km -> about PKR 120, 6 km -> about PKR 191, 12 km -> about PKR 332
```

CHECK

Checking Predictions by Hand

predict() is just the equation: intercept + coefficient x distance, with intercept = 49.9 and coefficient = 23.48.

Distance	By hand	model.predict()
3 km	49.9 + 23.48 x 3	PKR 120.4
6 km	49.9 + 23.48 x 6	PKR 190.8
12 km	49.9 + 23.48 x 12	PKR 331.7

INTERPRETATION

How to Read a Prediction

- A prediction of PKR 191 for 6 km means: rides like this typically cost about PKR 191. It is an estimate, not a promise
- Real fares scatter around the line. On the test rides, the average miss (MAE) was about PKR 21
- So a good way to say it: "About PKR 190, give or take 20"
- Stay inside the data. Our rides run from 0.5 to 26 km, so a prediction for a 100 km ride is extrapolation and cannot be trusted
- Predictions are only as good as the pattern in the data. If pricing rules change, retrain the model

QUICK CHECK

Quick Check

The model learns intercept = 50 and coefficient = 23.5. What fare would it predict for a 6 km ride?

ANSWER

Quick Check: Answer

```
predicted_fare = 50 + 23.5 * 6
print(predicted_fare)   # 191.0
#
# The model takes the base fare (50), adds the per-km rate
# (23.5) multiplied by the distance (6), and that sum is its
# prediction: roughly PKR 191.
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

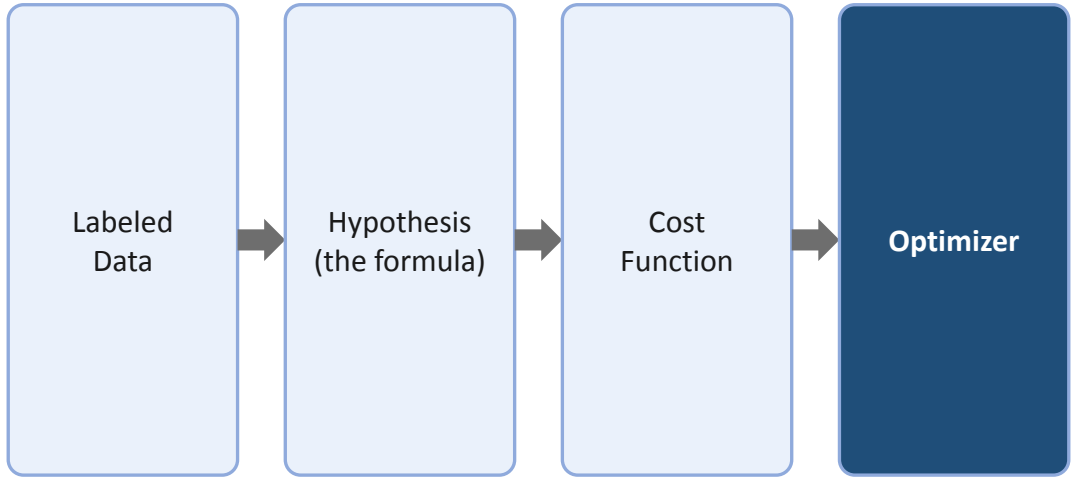
Hour 3

How the Model Is Fit, and How to Judge It

CONCEPT

Four Components of a Regression Model

- Labeled data: past rides where the fare is known
- Hypothesis (the model): the formula that turns features into a predicted fare
- Cost function: a single number measuring how wrong the predictions are
- Optimizer: the method that adjusts the coefficients to shrink the cost. For plain Linear Regression, scikit-learn solves this in one calculation



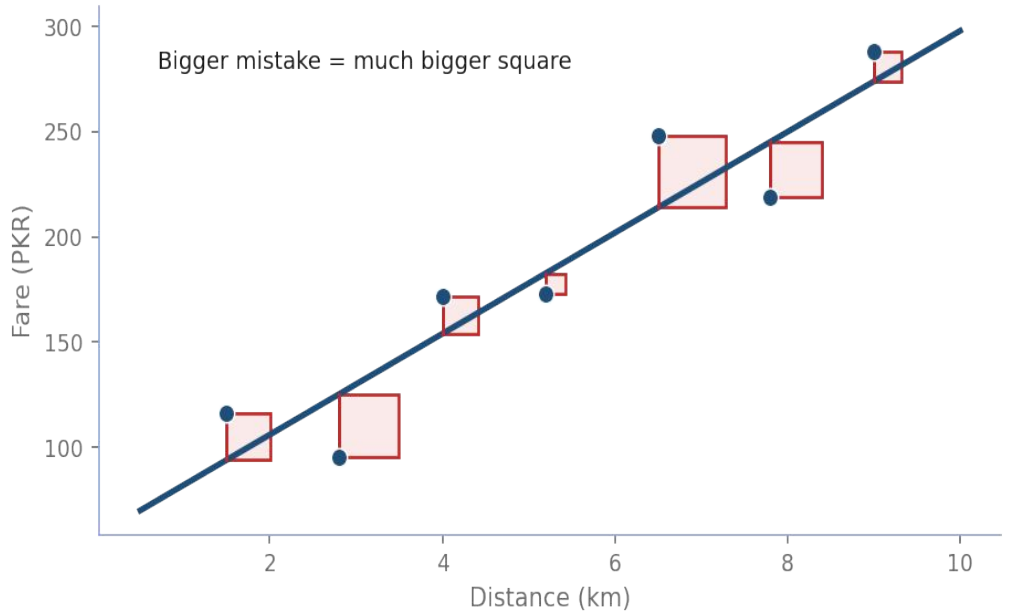
CONCEPT

Cost Function: Mean Squared Error

- For every ride, take the mistake (actual fare minus predicted fare) and square it
- Average all of those squares: that average is the MSE
- Squaring means one big mistake costs far more than several small ones
- A high MSE means the predictions are far from the actual fares

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

y_i = actual fare $\hat{y}_i$ = predicted fare n = number of rides



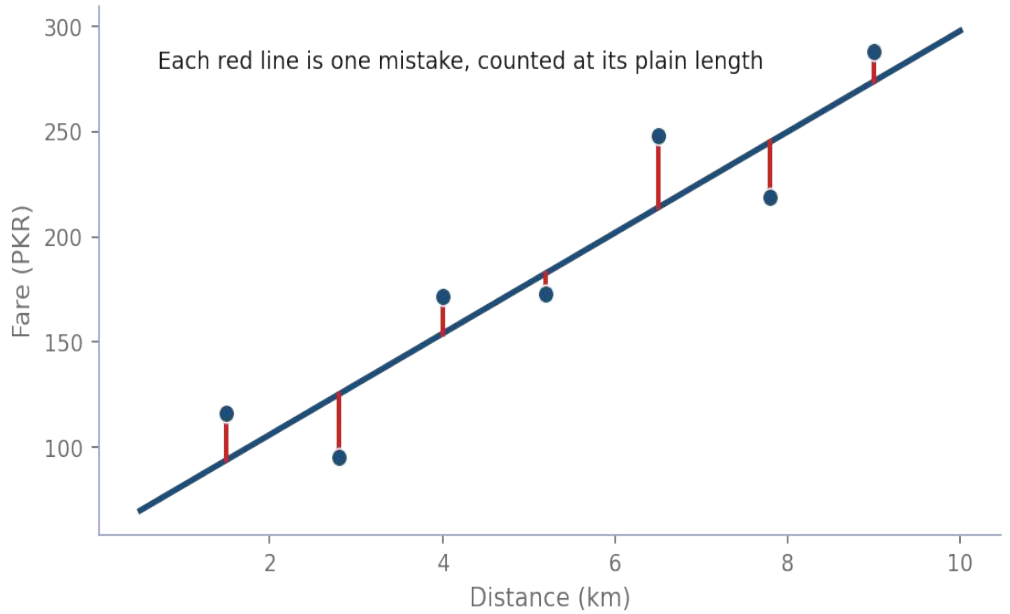
CONCEPT

Cost Function: Mean Absolute Error

- For every ride, take the mistake (actual fare minus predicted fare) and drop the minus sign: that is the absolute value
- Average all of those plain-size mistakes: that average is the MAE
- The answer is in PKR, so it reads directly: "on average we miss by about PKR 20"
- Less sensitive to big misses, so a few odd rides (outliers) will not dominate the score

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

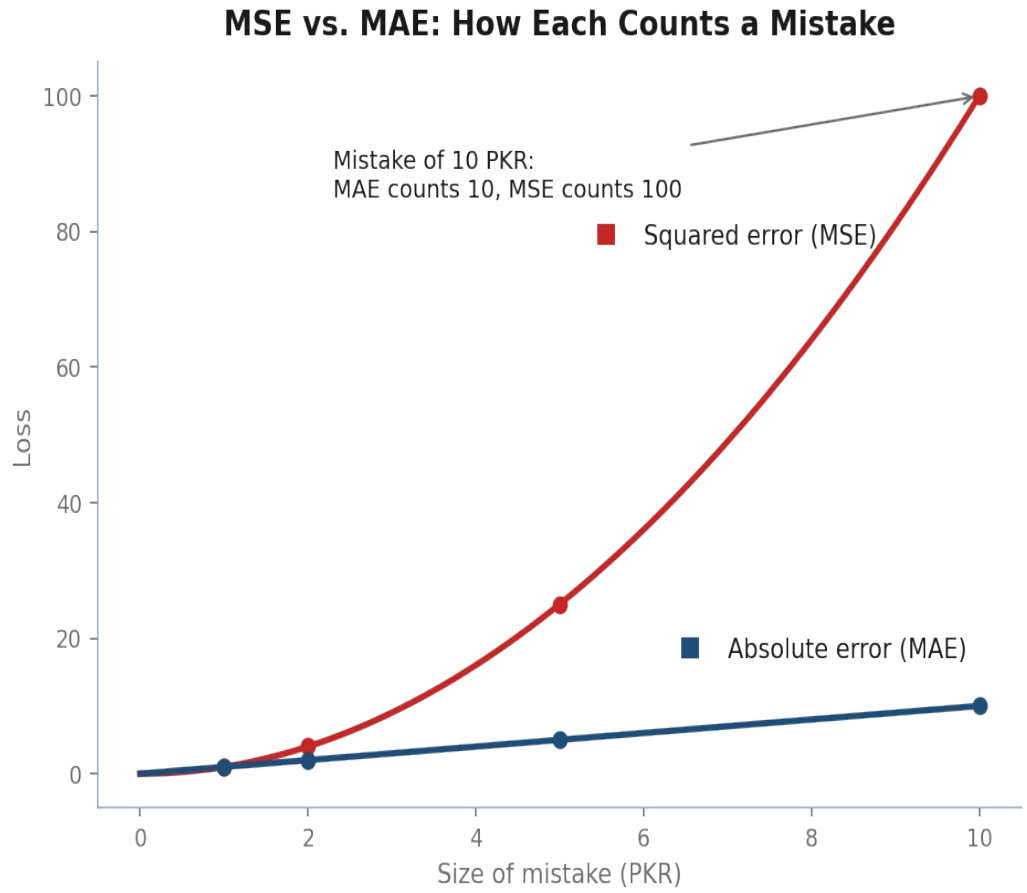
y_i = actual fare $\hat{y}_i$ = predicted fare n = number of rides



CONCEPT

MSE vs. MAE: Two Ways to Count a Mistake

- MAE averages the plain size of each mistake, so every PKR of error counts the same
- MSE squares them first: mistakes of 1, 2, 5, and 10 PKR become 1, 4, 25, and 100
- MSE is the better choice when big misses are especially costly
- MAE is more robust when a few odd rides (outliers) would otherwise dominate the score



CHECK

Checking MSE and MAE by Hand

Five test rides from ride_requests.csv. Error = actual minus predicted fare (PKR).

Actual	Predicted	Error	Absolute error	Squared error
133	122.7	10.3	10.3	106.0
160	174.4	-14.4	14.4	206.6
259	240.1	18.9	18.9	356.1
319	315.3	3.7	3.7	13.8
139	141.5	-2.5	2.5	6.2
		Average	MAE = 9.95	MSE = 137.7

CHECK

Checking MSE and MAE in Code

Calculate both by hand with NumPy, then confirm scikit-learn gives the same answers.

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error

errors = y_test - predictions

mae_by_hand = np.mean(np.abs(errors))   # drop the minus signs, average
mse_by_hand = np.mean(errors ** 2)      # square, average

print(mae_by_hand, mean_absolute_error(y_test, predictions))
print(mse_by_hand, mean_squared_error(y_test, predictions))

# Each pair prints the same number. On all 200 test rides:
# MAE = 20.97, MSE = 702.5, RMSE = 26.5
```

INTERPRETATION

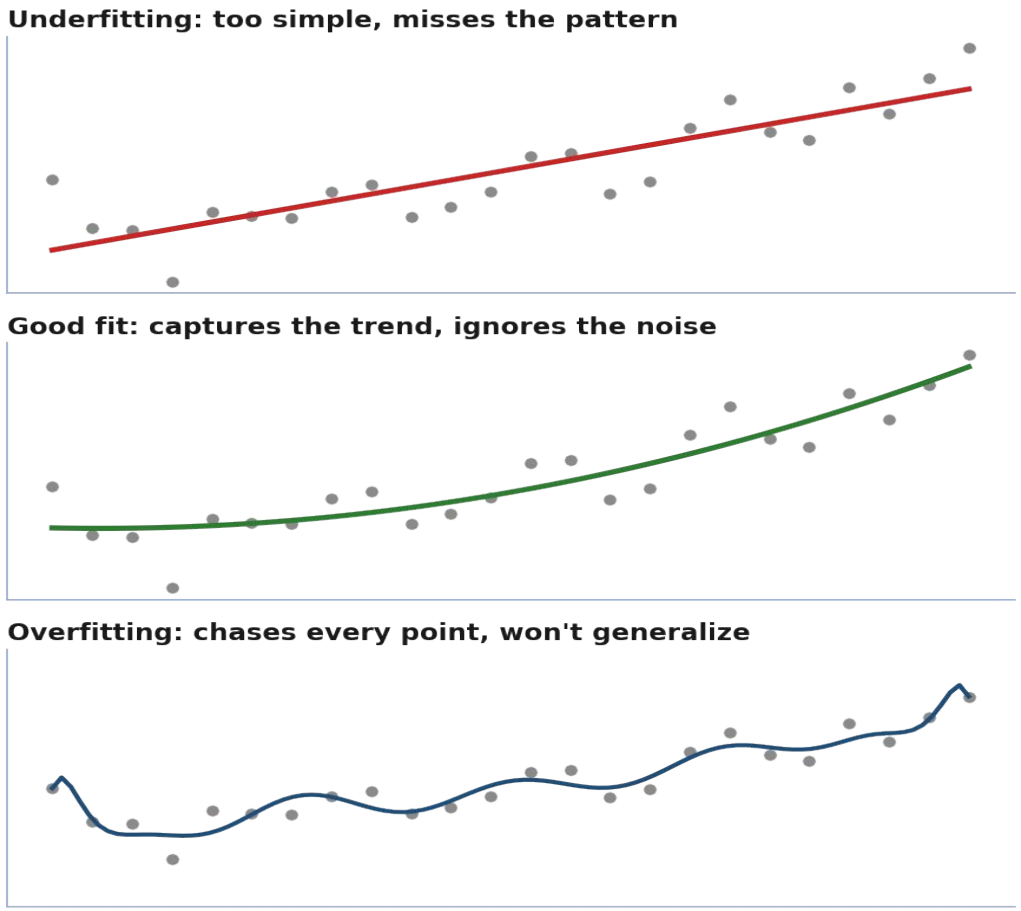
MAE vs MSE on the Same Rides

- MAE of 9.95 on these five rides: on average the prediction misses by about PKR 10
- MSE of 137.7 is in squared PKR, so it is hard to picture. Its square root, the RMSE, is about PKR 11.7
- The biggest miss (18.9) is 38% of the total absolute error (18.9 of 49.8), but 52% of the total squared error (356.1 of 688.7)
- That is why MSE reacts strongly to a few big mistakes and MAE does not

CONCEPT

Underfitting, Good Fit & Overfitting

- A model that's too simple misses real patterns in the data: underfitting
- A model that's too flexible memorizes the training rides exactly, noise included, and fails on new ones: overfitting
- The goal is always the middle ground: a model that captures the real trend, not every bump in the training data
- This tension shows up in every model this month, from today's line to Friday's neural network



CONCEPT

Simple vs. Multiple Regression

- Simple Linear Regression uses one feature (distance) to predict fare
- Multiple Linear Regression uses several features at once: distance, time_of_day, area_type together
- Categorical columns like time_of_day can't be multiplied directly, they're one-hot encoded into separate 0/1 columns first
- More relevant features usually means a more accurate estimator, up to a point, adding useless features can hurt instead of help

EXAMPLE

One-Hot Encoding in Code

pd.get_dummies turns every text column into 0/1 columns. We pass only the feature columns on purpose.

```
import pandas as pd

rides = pd.read_csv('ride_requests.csv')

features = pd.get_dummies(
    rides[['distance_km', 'time_of_day', 'area_type']],
    drop_first=True,
    dtype=int,
)
print(features.head())

# fare_pkr is the target, so it stays OUT of the features.
# Leaving it in would hand the model the answer (data leakage).
```

RESULT

Before and After get_dummies

First three rides of ride_requests.csv. Numbers pass through, text becomes 0/1 columns.

Original ride	distance_km	Evening	Morning	Night	Commercial	Residential
Afternoon, Commercial	7.0	0	0	0	1	0
Afternoon, Residential	9.0	0	0	0	0	1
Night, Commercial	6.3	0	0	1	1	0

CONCEPT

Why drop_first=True?

- Each ride is in exactly one area, so the dummy columns always add up to 1
- That makes one column redundant: if Commercial and Residential are both 0, the ride must be Airport
- A linear model with an intercept cannot tell the redundant column apart from the intercept, so its coefficients become ambiguous
- Dropping one category fixes this. Its information is not lost: it is the case where all remaining columns are 0
- This matters for linear-type models (Linear and Logistic Regression), not for decision trees

Keep all three: every row adds up to 1

ride	Airport	Commercial	Residential	Sum
Airport ride	1	0	0	**1**
Commercial ride	0	1	0	**1**
Residential ride	0	0	1	**1**

Any one column can be worked out from the other two: redundant

drop_first=True: Airport is left out

ride	Commercial	Residential
Airport ride	0	0
Commercial ride	1	0
Residential ride	0	1

Both 0 = Airport (the baseline)

CONCEPT

The Dropped Category Is the Baseline

It gets no coefficient of its own. Its effect lives in the intercept, and every other coefficient is a shift from it.

Ride	Dummies switched on	Equation (real model)
Afternoon, Airport (baseline)	None, all 0	fare = 60 + 23.7 x distance
Evening, Airport	Evening	fare = 60 + 32.8 + 23.7 x distance
Afternoon, Commercial	Commercial	fare = 60 - 29.8 + 23.7 x distance
Evening, Commercial	Evening, Commercial	fare = 60 + 32.8 - 29.8 + 23.7 x distance

CHECK

Drop vs Do Not Drop: Same Predictions, Different Coefficients

Real fit on ride_requests.csv with distance_km and area_type. The first three rides get the same predicted fares either way (210.6, 255.9, 194.1).

	Intercept	distance_km	Airport	Commercial	Residential
drop_first=True	73.9	23.6	(dropped)	-28.4	-30.3
No drop	54.3	23.6	+19.6	-8.8	-10.7

INTERPRETATION

Reading the Two Fits

- With the drop: Commercial is about PKR 28 cheaper than Airport, a clear, readable comparison
- Without the drop: Airport +19.6 and Commercial -8.8 do not mean "adds 19.6" or "takes off 8.8". The intercept and the dummies share the baseline arbitrarily
- What survives either way is the gap between categories: 19.6 - (-8.8) = 28.4, the same gap
- The choice of which category to drop changes how coefficients read, never the predictions
- Rule of thumb: drop one category whenever you want to read the coefficients of a linear model

EXAMPLE

A Multi-Feature Fare Estimator

pd.get_dummies turns category columns into numbers a model can use.

```
features = pd.get_dummies(
    rides[['distance_km', 'time_of_day', 'area_type']],
    drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    features, rides['fare_pkr'], test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

print(dict(zip(features.columns, model.coef_)))
```

INTERPRETATION

Reading a Multi-Feature Model

Real output from ride_requests.csv. Baselines are Afternoon and Airport, the categories dropped by drop_first=True.

Term	Value	What it means
Intercept	60	Baseline fare: an Afternoon airport ride of 0 km (a reference point, not a real ride)
distance_km	+24	Each extra kilometer adds about PKR 24 (23.7), other features held constant
time_of_day_Evening	+33	An evening ride costs about PKR 33 more than the same ride in the afternoon
area_type_Commercial	-30	A commercial-area ride costs about PKR 30 less than an airport ride
area_type_Residential	-32	A residential ride costs about PKR 32 less than an airport ride

EXAMPLE

Predicting With Several Features

The new ride needs the same columns the model was trained on, so we build one row with the 0/1 dummy columns.

```
new = pd.DataFrame([dict.fromkeys(features.columns, 0)] * 2)

# Ride A: 4 km, Evening, Commercial area
new.loc[0, ['distance_km', 'time_of_day_Evening', 'area_type_Commercial']] = [4, 1, 1]

# Ride B: 4 km, Afternoon, Residential area
new.loc[1, ['distance_km', 'area_type_Residential']] = [4, 1]

print(model.predict(new))   # [157.9  122.9]
```

What new looks like after the two .loc lines:

	distance_km	time_of_day_Evening	time_of_day_Morning	time_of_day_Night	area_type_Commercial	area_type_Residential
0 (Ride A)	4	1	0	0	1	0
1 (Ride B)	4	0	0	0	0	1

model.predict(new) gives one fare per row: Ride A = PKR 157.9, Ride B = PKR 122.9

INTERPRETATION

Same Distance, Different Fare

- Ride A (4 km, Evening, Commercial) is predicted at about PKR 158. Ride B (4 km, Afternoon, Residential) at about PKR 123
- Check A by hand: 60.3 + 23.7 x 4 + 32.8 (Evening) - 29.8 (Commercial) = 157.9
- The 35 rupee gap comes entirely from time of day and area, since the distance is identical
- A distance-only model would give both rides the same fare (about PKR 144). The extra features make it more realistic

CONCEPT

Why This Matters

- Coefficients quantify each feature's impact, which makes the model a source of business insight, not just predictions
- You can see which features matter most, and which barely matter at all
- This guides feature selection, pricing policy, and forecasting
- A fare model that can explain itself is far easier to trust, audit, and defend than a black box

REFERENCE

Judging a Regression Model

Four common ways to score how close predictions land to actual fares.

Metric	What It Measures	How to Read It
MAE	Mean Absolute Error, average size of the mistake	In PKR, same units as fare
MSE	Mean Squared Error, punishes big mistakes harder	In squared PKR, harder to read directly
RMSE	Square root of MSE	Back in PKR, easier to interpret than MSE
R^2	Share of fare's variation the model explains	0 to 1, closer to 1 is better

EXAMPLE

Scoring the Fare Estimator

All four metrics, computed on the held-out test rides.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

predictions = model.predict(X_test)

print('MAE:', mean_absolute_error(y_test, predictions))
print('RMSE:', np.sqrt(mean_squared_error(y_test, predictions)))
print('R2:', r2_score(y_test, predictions))
```

INTERPRETATION

Reading the Scores: One Feature vs Several

Test-set results on ride_requests.csv (random_state=42).

Model	MAE	RMSE	R^2
Distance only	PKR 21.0	PKR 26.5	0.90
Distance + time of day + area	PKR 18.7	PKR 23.7	0.92

INTERPRETATION

What the Scores Tell Us

- MAE of PKR 21: on average, the prediction misses the real fare by about PKR 21
- RMSE is higher than MAE (26.5 vs 21.0) because it punishes the occasional large miss harder
- R^2 of 0.90: distance alone explains about 90% of the ups and downs in fare. Adding time and area lifts that to 0.92
- Lower errors and a higher R^2 on the TEST set means the extra features genuinely help, not just memorise the training data
- Always ask: is a PKR 19 average miss good enough for the app? That is a business decision, not a statistics one

RECAP

Steps in a Regression Problem

- 1. Data collection: gather numeric features and a numeric target
- 2. Preprocessing: handle missing values, one-hot encode categories, scale if needed
- 3. Feature selection: keep the features that genuinely affect the target
- 4. Model training: fit the regression model on the training set
- 5. Evaluation: check MAE, RMSE, and R^2 on the test set
- 6. Prediction: estimate the fare for brand-new rides

PRACTICE

Practice

Using ride_requests.csv, fit a multi-feature fare estimator (distance_km, time_of_day, area_type). Report its MAE and R^2 on the test set, read its coefficients aloud in plain language, then predict the fare for your own made-up ride: 9.5 km, Evening, Commercial area.

Key Takeaways & Coming Up

- Supervised learning learns from labeled examples. Regression predicts a number, classification predicts a category
- Every ML project follows the same workflow: problem, data, features, split, train, evaluate, improve
- Linear Regression fits the best straight line, and its coefficients are readable in plain language
- Your fare estimator is the first piece of this week's project: a full ride-hailing prediction engine
- Coming up next: can we also predict whether a ride gets picked up quickly? That's a classification problem